# EXPERIMENT - 47

## 1. Problem Statement

Write a C program that reads the size and elements of a sorted variable-length integer array, arr[], in ascending order. Also, read an integer target. Pass the array to a function named ceil_index() using its address. Use pointers to pass the array between the calling function and ceil_index(). The function ceil_index() must:

- Find the index of the smallest element in arr[] that is greater than or equal to target.
- Treat this element as the ceil of target.
- If multiple occurrences of the ceil element exist, return the index of its first occurrence.
- Display the ceil element and its index.
- Display -1 if no element is greater than or equal to target.

For 5 7 7 8 8 10 with target 6 the output is Ceil of 6: 7 at index 1. For 5 7 7 8 9 10 with target 8 it is Ceil of 8: 8 at index 3. For 5 7 7 8 8 10 with target 11 it is -1.

## 2. Algorithm

### Main Program

**Step 1:** START

**Step 2:** Read n and declare the variable length array arr of size n.

**Step 3:** Read the n elements of arr in ascending order.

**Step 4:** Read target.

**Step 5:** Call ceil_index(&arr[0], n, target).

**Step 6:** STOP

### Function ceil_index()

**Step 1:** Repeat for i = 0 to n − 1:

If arr[i] is greater than or equal to target, display target, arr[i] and i, then return to the calling function.

**Step 2:** Display −1, since no element is greater than or equal to target.

**Step 3:** Return to the calling function.

## 3. Test Case Table

| # | Array | Target | Expected Output | Actual Output | Result |
|---|---|---|---|---|---|
| 1 | 5 7 7 8 8 10 | 6 | Ceil of 6: 7 at index 1, the first 7 and not the second | Matches example 1 in the question | **PASS** |
| 2 | 5 7 7 8 9 10 | 8 | Ceil of 8: 8 at index 3, the target itself | Matches example 2 | **PASS** |
| 3 | 5 7 7 8 8 10 | 11 | −1, nothing is that big | −1 | **PASS** |

| # | Array | Target | Expected Output | Actual Output | Result |
|---|---|---|---|---|---|
| 4 | 5 7 7 8 8 10 | 7 | Index 1, since 7 appears twice and the first one wins | Index 1, not 2 | **PASS** |
| 5 | 5 7 7 8 8 10 | 5 | Ceil of 5: 5 at index 0, the first element | 5 at index 0 | **PASS** |
| 6 | 5 7 7 8 8 10 | 10 | Ceil of 10: 10 at index 5, the last element | 10 at index 5 | **PASS** |
| 7 | 5 7 7 8 8 10 | 2 | Ceil of 2: 5 at index 0, as the target is below everything | 5 at index 0 | **PASS** |

## 4. Source Code

### ceil47.c

```
/* Experiment 42. One function for each of the three operations.
   Every option reads its own input here, which is different from what
   I have been doing. Option 3 needs two strings and the other two need
   one, so there was no single string I could read at the start that
   would have suited all three.
   The question lists only three operations, but a do while menu has no
   way of stopping, so option 4 is mine. Without it the menu keeps
   coming back and the only way out is to kill the program. */

#include <stdio.h>

void removeVowels(const char s[], char result[]);
char firstRepeatingLowercase(const char s[]);
int areAnagrams(const char s1[], const char s2[]);

int main(void)
{
    char str[100], str2[100], result[100];
    char ch;
    int choice;

    do {
        printf("\nMENU\n");
        printf("1. Remove all the vowels from a string\n");
        printf("2. Find the first repeating lowercase letter\n");
        printf("3. Check whether two strings are anagrams\n");
        printf("4. Exit\n");
        printf("Enter your choice: ");
        scanf("%d", &choice);

        switch (choice) {
            case 1:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                removeVowels(str, result);
                printf("String without vowels: %s\n", result);
                break;

            case 2:
                printf("Enter a string: ");
                scanf(" %[^\n]", str);
                ch = firstRepeatingLowercase(str);
                if (ch == '\0')
                    printf("No lowercase letter repeats in this string\n");
                else
                    printf("First repeating lowercase letter: %c\n", ch);
                break;

            case 3:
                printf("Enter the first string: ");
                scanf(" %[^\n]", str);
                printf("Enter the second string: ");
                scanf(" %[^\n]", str2);
                if (areAnagrams(str, str2))
                    printf("The two strings are anagrams\n");
                else
                    printf("The two strings are not anagrams\n");
                break;

            case 4:
                printf("Exiting the program.\n");
                break;
        }
    } while (choice != 4);

    return 0;
}
```

```
/* i walks through the old string and j marks where to write in the new
   one, and they drift apart because j only moves when something is
   actually copied.
   The '\0' on the last line is the one I forgot. Programming in C came
   out right and then had junk stuck on the end of it, because printf
   had no idea where the new string finished.
   Both cases are listed because A and a are different characters, and
   leaving the capitals out would have let the A through in Day. */
void removeVowels(const char s[], char result[])
{
    int i, j = 0;
    char ch;

    for (i = 0; s[i] != '\0'; i++) {
        ch = s[i];

        if (ch != 'a' && ch != 'e' && ch != 'i' && ch != 'o' && ch != 'u' &&
            ch != 'A' && ch != 'E' && ch != 'I' && ch != 'O' && ch != 'U') {
            result[j] = ch;
            j++;
        }
    }

    result[j] = '\0';
}

/* First repeating can be read two ways, and the two give different
   answers. In abcdbca one reading says b, the first letter you meet
   that you have seen before. The other says a, because out of the
   letters that repeat at all, a is the one that showed up earliest.
   I went with the second, which is the usual meaning, and that is why
   the counting happens first and the walk from the left happens after.
   One single pass would have given the other answer.
   count has 26 boxes, one per small letter, and s[i] - 'a' turns a
   letter straight into its box number, since 'a' is 97 and 'b' is 98.
   Capitals are skipped in both loops, so AABBcdc answers c, even though
   A and B repeat more times than c does. */
char firstRepeatingLowercase(const char s[])
{
    int count[26] = {0};
    int i;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z')
            count[s[i] - 'a']++;

    for (i = 0; s[i] != '\0'; i++)
        if (s[i] >= 'a' && s[i] <= 'z' && count[s[i] - 'a'] > 1)
            return s[i];

    /* '\0' is not a letter, so main can use it to mean nothing was
       found. */
    return '\0';
}

/* Anagrams hold the same characters the same number of times, only in
   a different order. My first idea was to sort both strings and see if
   they came out identical, but counting turned out to be less code, and
   it leaves the two strings alone instead of rearranging them.
   256 boxes covers every character there is, so spaces and digits are
   counted along with the letters, and nothing is quietly ignored.
   The length test is not only a shortcut. The counts could agree on
   every character and the strings still be different lengths if one of
   them held something the other did not. */
int areAnagrams(const char s1[], const char s2[])
{
    int count1[256] = {0};
    int count2[256] = {0};
    int i, len1 = 0, len2 = 0;

    for (i = 0; s1[i] != '\0'; i++) {
        count1[(unsigned char) s1[i]]++;
        len1++;
```

```
    }

    for (i = 0; s2[i] != '\0'; i++) {
        count2[(unsigned char) s2[i]]++;
        len2++;
    }

    if (len1 != len2)
        return 0;

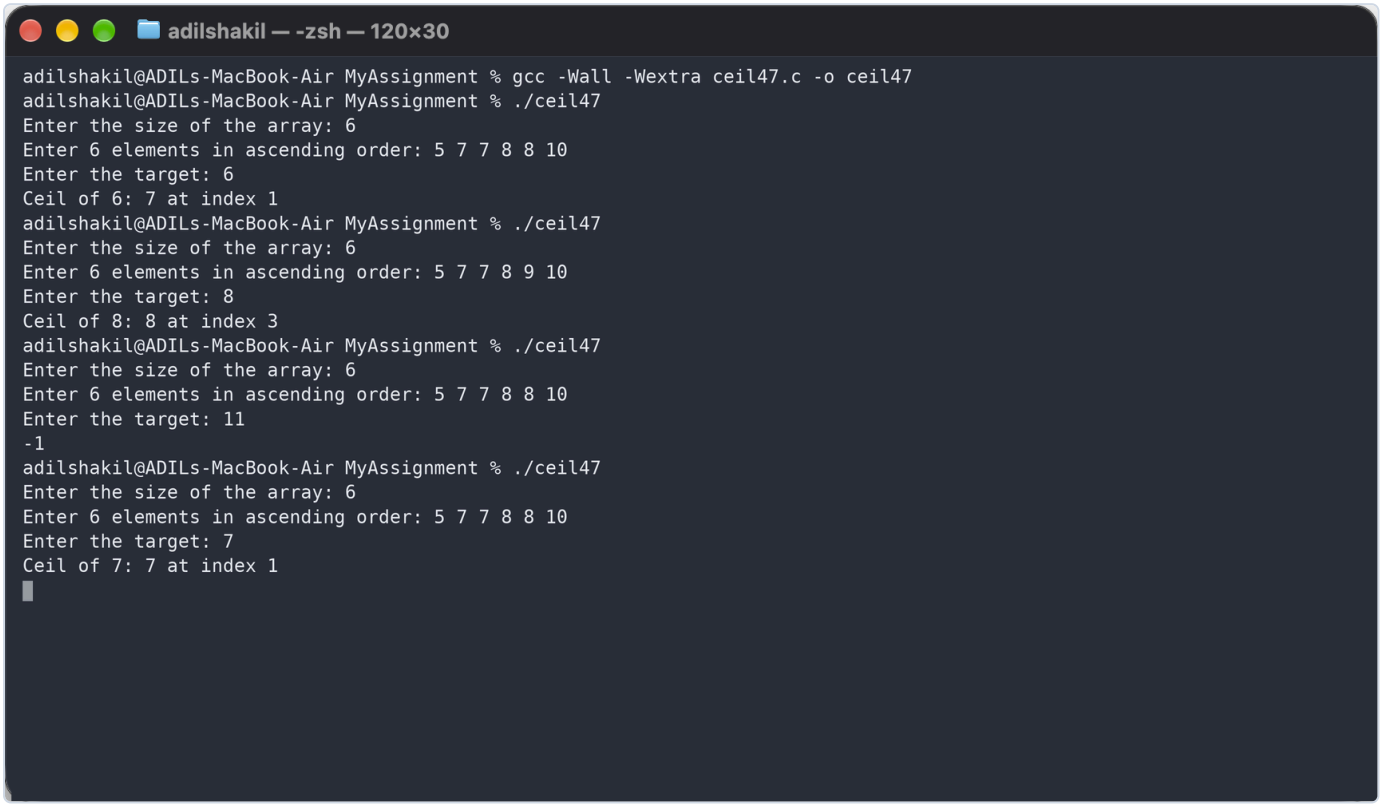
    for (i = 0; i < 256; i++)
        if (count1[i] != count2[i])
            return 0;

    return 1;
}
```

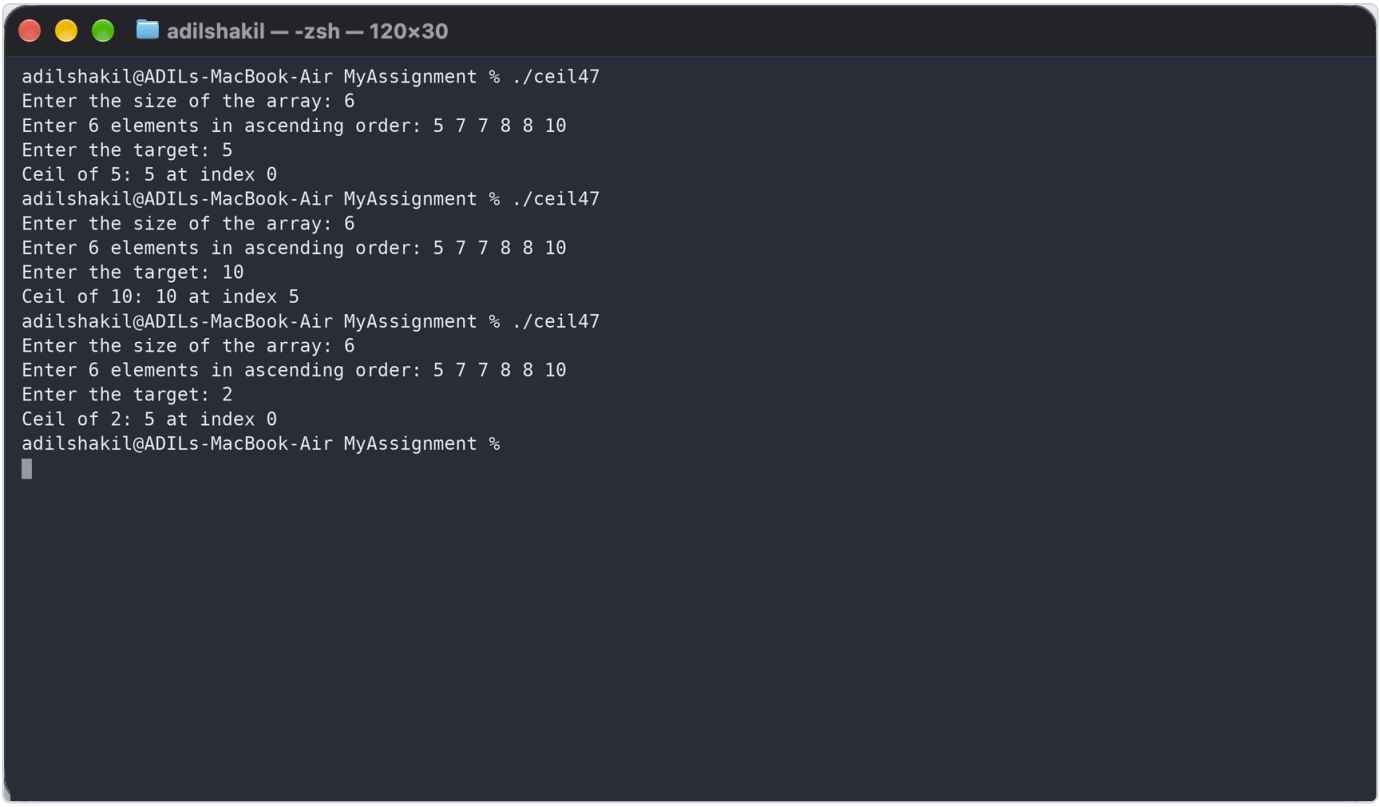
## 5. Compilation and Output

Compiled with `gcc -Wall -Wextra ceil47.c -o ceil47` so the small warnings would show up too. It built clean.

### The Three Examples from the Question, and a Repeated Ceil



### The Target at Both Ends, and a Target Below the Whole Array




All seven cases came out right, and the first three are the examples given in the question.

The nice part here is that the question asks for two things and one loop gives both. Walking from the left, the first element that is not smaller than the target is the smallest such element, so it is the ceil. And because the array is sorted, any earlier copy of that same value would have been met first, so the index found is already the first occurrence. No extra check for duplicates is needed. Case 4 is the one that proves it: target 7 gives index 1, not 2.

ceil_index() takes `int *arr`, so it is handed the address of the array rather than a copy of it, which is what the question asks for. The call is written as `&arr[0]`, the address of the first element.

The `return` inside the loop is doing more than it looks. It leaves the function the moment a match is printed, which is what stops the −1 line below from running as well. Without it the program would print the answer and then print −1 underneath it.

Cases 5, 6 and 7 I added myself: the target sitting on the first element, on the last one, and below the whole array. The last of those is the one people forget, because the answer is still a valid ceil and not −1.